PersonalFinanceManager

COMPLETE PROJECT FLOW — FROM IDEA TO WORKING APPLICATION

Technology: SQL Server + C# + ADO.NET + ASP.NET Core Web API + ASP.NET Core MVC

Purpose: Build a personal finance manager for your own monthly income, spending, savings, investments, bills and goals.

Stage	Main Result
1. Requirement	Know exactly what the application must do
2. Database	SQL Server schema, relationships and queries
3. C# / Models	Objects representing database/application data
4. ADO.NET	Application communicates with SQL Server
5. Web API	Business/data operations exposed as HTTP endpoints
6. MVC	Web pages, forms and dashboard
7. Business Logic	Budget, savings and plan-vs-actual calculations
8. Testing	Verify every flow
9. Deployment	Run the complete application

This flow keeps the source PDF's Phase-2 database-design discipline—objectives, rules, entity groups, data dictionary, relationships, SQL and implementation planning—but applies it to the personal finance project.

1. Project Requirement — What Are We Building?

The application starts from a real personal situation: you receive a monthly salary, decide how you want to use it, record what actually happens, and review the result.

Example: Monthly income = ₹50,000.

Money Area	Example Amount	Purpose
Income	₹50,000	Salary received
Rent / Fixed Needs	₹10,000	Necessary expenses
Food	₹5,000	Daily food/groceries
Travel	₹3,000	Transport
Bills	₹2,000	Internet/mobile/electricity
Savings	₹10,000	Emergency or goal
Investment	₹5,000	SIP/stocks/etc.
Flexible / Remaining	₹15,000	Your own choice

The application does not force a fixed percentage rule. You enter your own plan, and the system compares your plan with your actual transactions.

2. Complete Functional Flow

Step	What You Do	What System Does
1	Login	Authenticates the user
2	Enter income	Stores salary/other income
3	Add accounts	Stores HDFC/Cash/Wallet/etc.
4	Create categories	Food/Rent/Travel/Savings/etc.
5	Create monthly plan	Stores your planned amount per category
6	Record expense	Stores actual transaction
7	Record savings	Updates savings goal progress
8	Record investment	Stores invested/current value
9	Add bills	Tracks due date and payment status
10	Open dashboard	Calculates income, expenses, savings, investment and remaining
11	Review reports	Shows category/month/account trends

3. Complete Technical Architecture

The application is split into responsibilities so that you learn how a real .NET application is organized.

Layer	Responsibility	Example
Browser	User interface	Dashboard / Add Expense
ASP.NET Core MVC	Web UI flow	Controller + ViewModel + View
Web API	HTTP service layer	GET/POST/PUT/DELETE endpoints
Service Layer	Business rules	Calculate monthly balance
Repository	Database access	ADO.NET methods
ADO.NET	SQL communication	SqlConnection / SqlCommand / Reader
SQL Server	Persistent data	Tables / FK / Stored Procedures

Typical request: Browser → MVC Controller → Service → Repository → ADO.NET → SQL Server → back through the same layers → View

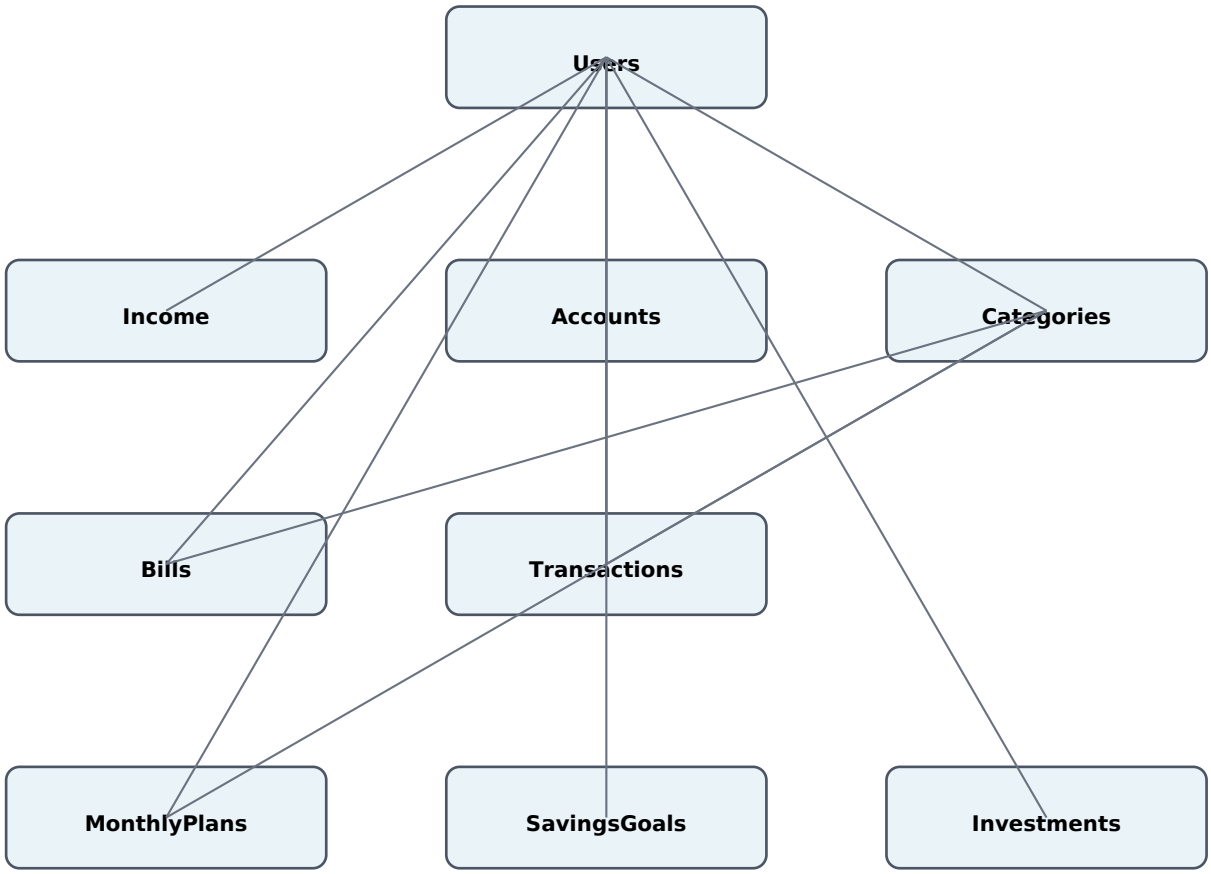
4. Database Flow

The core database is intentionally smaller than the ERP reference because this is a personal application.

Table	Why It Exists	Main Relationship
Users	Application owner/login	Parent of personal records
Income	Salary and other earnings	Users 1:N Income
Accounts	Where money is held	Users 1:N Accounts
Categories	Classifies money	Users 1:N Categories
Transactions	Actual daily money movement	Users/Accounts/Categories
MonthlyPlans	Your monthly allocation	Users + Categories
SavingsGoals	Target-based saving	Users 1:N Goals
Investments	Investment tracking	Users 1:N Investments
Bills	Recurring obligations	Users + Categories

The source PDF uses the same database-design idea of parent-child relationships and 1:N cardinality for its ERP entities; here the parent is mainly the personal user. □filecite□turn0file0□L425-L468□

5. ER Diagram — Main Flow



Read it like this: One User can have many Accounts. One Account can have many Transactions. One Category can be used by many Transactions and MonthlyPlans.

6. SQL Server → ADO.NET Flow

When you click Save Expense, the data travels through several layers.

```
1. User fills form 2. MVC Controller receives model 3. Controller calls Service 4. Service validates business rules 5. Service calls Repository 6. Repository creates SqlConnection 7. Repository creates SqlCommand 8. Parameters are added 9. Stored Procedure / SQL executes 10. SQL Server inserts the row 11. Result returns to Repository 12. Service returns result 13. Controller redirects to Dashboard
```

ADO.NET Class	Purpose
SqlConnection	Opens connection to SQL Server
SqlCommand	Runs SQL or stored procedure
SqlParameter	Safely passes values to SQL
SqlDataReader	Reads rows returned by SELECT
SqlDataAdapter	Can fill DataSet/DataTable when needed
SqlTransaction	Groups multiple database operations atomically

7. Web API Flow

The Web API gives the application a clean HTTP interface. MVC can consume it, and later a mobile or JavaScript client can use the same API.

HTTP	Endpoint Example	Purpose
POST	/api/income	Create income
GET	/api/income	Get income list
GET	/api/income/{id}	Get one income
PUT	/api/income/{id}	Update income
DELETE	/api/income/{id}	Delete/cancel where allowed
POST	/api/transactions	Create expense/income transaction
GET	/api/transactions	List transactions
GET	/api/dashboard	Get dashboard summary
GET	/api/reports/monthly	Monthly report

```
HTTP Request ↓ API Controller ↓ DTO Validation ↓ Service ↓ Repository ↓ ADO.NET ↓ SQL Server ↓ Response DTO ↓ JSON Response
```

8. ASP.NET Core MVC Flow

MVC is responsible for the browser-facing application.

MVC Part	Responsibility	Example
Model / DTO	Data shape	ExpenseCreateDto
Controller	Receives browser request	TransactionsController
Service	Business calculation	TransactionService
Repository	Database operations	TransactionRepository
View	HTML user interface	Create.cshtml
ViewModel	Data needed by a screen	DashboardViewModel

Add Expense screen:

```
Create.cshtml ↓ POST TransactionsController.Create() ↓ TransactionService.AddExpense() ↓ TransactionRepository.Insert() ↓
ADO.NET ↓ SQL Server ↓ Redirect to Dashboard
```

9. CRUD Development Order

Order	Module	CRUD to Practice
1	Categories	Create / Read / Update / Deactivate
2	Accounts	Create / Read / Update / Deactivate
3	Income	Create / Read / Update
4	Transactions	Create / Read / Update / controlled delete
5	MonthlyPlans	Create / Read / Update
6	SavingsGoals	Create / Read / Update
7	Investments	Create / Read / Update
8	Bills	Create / Read / Update / mark paid
9	Dashboard	Read/calculations
10	Reports	Read/group/filter/export later

Do not start with the dashboard. Build the data-entry CRUD first; the dashboard becomes much easier once reliable data exists.

10. Business Logic — What the Application Calculates

Calculation	Formula
Total Income	SUM(Income.Amount) for selected month
Total Expense	SUM(Transactions.Amount) where TransactionType = Expense
Remaining	Your defined income minus actual allocations/expenses
Category Actual	SUM(Expense) grouped by Category
Plan Difference	PlannedAmount − ActualAmount
Goal Remaining	TargetAmount − CurrentAmount
Goal Progress	CurrentAmount / TargetAmount × 100
Investment Gain/Loss	CurrentValue − InvestedAmount
Bill Status	Paid / Unpaid / Due Soon based on date and IsPaid

Business rules should live in the Service layer rather than being scattered across MVC controllers and views.

11. Complete Example — ₹50,000 Monthly Salary

This is the practical flow you can follow while testing the project.

Day / Action	Record	Result
1. Salary received	Income ₹50,000	Monthly income becomes ₹50,000
2. Plan created	Food ₹5,000; Rent ₹10,000; etc.	Monthly plan saved
3. Rent paid	Expense ₹10,000	Transaction recorded
4. Food purchase	Expense ₹500	Food actual = ₹500
5. SIP	Investment ₹5,000	Investment record updated
6. Savings	Goal contribution ₹10,000	Goal progress updated
7. Electricity	Bill ₹1,200	Bill marked paid
8. Dashboard	Open dashboard	Plan vs actual displayed

Example dashboard after these operations:

Metric	Value
Income	₹50,000
Recorded Expenses	₹11,700
Savings Goal Contribution	₹10,000
Investment	₹5,000
Remaining	Calculated from your transaction design

12. Stored Procedure Flow

For ADO.NET practice, create stored procedures for important operations.

```
CREATE PROCEDURE sp_AddTransaction @UserId INT, @AccountId INT, @CategoryId INT, @Amount DECIMAL(18,2), @TransactionType
NVARCHAR(20), @TransactionDate DATETIME2, @Description NVARCHAR(500) AS BEGIN INSERT INTO Transactions ( UserId, AccountId,
CategoryId, Amount, TransactionType, TransactionDate, Description ) VALUES ( @UserId, @AccountId, @CategoryId, @Amount,
@TransactionType, @TransactionDate, @Description ); END;
```

ADO.NET calls it with SqlCommand.CommandType = CommandType.StoredProcedure and parameterized values.

13. Validation and Error Handling Flow

```
Browser validation ↓ MVC Model Validation ↓ Service business validation ↓ Repository/database constraints ↓ Exception handling ↓ Logging ↓ User-friendly error response
```

Examples: amount must be greater than zero; month must be 1–12; category must belong to the user; an inactive account should not accept a new transaction; financial records should not be casually deleted.

14. Authentication and Security Flow

```
Login ↓ Email + Password ↓ Password verification ↓ Authentication Cookie / Token ↓ Authenticated request ↓ Current UserId ↓ Every query filters by UserId ↓ Return only that user's data
```

For a personal application, this still gives you valuable practice with authentication and authorization. Never store a plain-text password; store a secure password hash.

15. Testing Flow

Test Area	Example
Database	Insert user; verify FK; verify constraints
ADO.NET	Connection opens/closes correctly
Repository	Correct row inserted/read
Service	Invalid amount rejected
API	POST returns correct status/response
MVC	Form validation works
Security	User cannot read another user's records
Dashboard	Income/expense totals are correct
Plan vs Actual	Over-budget categories are detected
Bills	Paid/unpaid status is correct

Test with known numbers. For example, enter ₹50,000 income and a small set of expenses where you can manually calculate the expected result before checking the dashboard.

16. Recommended Solution Structure

```
PersonalFinanceManager.sln ├── PersonalFinanceManager.Domain │ ├── Entities │ └── DTOs / Models │ ├──
PersonalFinanceManager.Application │ ├── Interfaces │ ├── Services │ └── DTOs │ ├── PersonalFinanceManager.Infrastructure │
├── Data │ ├── Repositories │ └── ADO.NET │ ├── PersonalFinanceManager.Api │ ├── Controllers │ ├── Middleware │ └── Program.cs
│ └── PersonalFinanceManager.Mvc ├── Controllers ├── Views ├── ViewModels ├── wwwroot └── Program.cs
```

This is a learning-oriented layered architecture. You can simplify it during the first lessons, then grow into this structure as the project becomes larger.

17. 20-Day Learning + Project Flow

Day	Learn / Build
1	Project requirement + SQL Server basics
2	Create database + Users + Accounts
3	Categories + relationships + FK
4	Income + Transactions
5	SELECT + WHERE + ORDER BY + JOIN
6	GROUP BY + SUM + monthly reports
7	Stored procedures + constraints + indexes
8	C# models/classes/properties
9	ADO.NET connection + SqlCommand
10	ADO.NET CRUD + repository
11	Web API project + first endpoint
12	Web API CRUD + DTOs
13	MVC project + routing + controllers
14	MVC Views + forms + validation
15	Income/Account/Category/Transaction CRUD
16	MonthlyPlans + Plan vs Actual
17	SavingsGoals + Investments + Bills
18	Dashboard + reports
19	Authentication + exception handling + logging
20	Testing + cleanup + final project review

18. Final End-to-End Flow

```
YOU ↓ Open Personal Finance Manager ↓ LOGIN ↓ DASHBOARD ↓ Enter Monthly Income ↓ Create Monthly Plan ↓
┌──────────────────────────────────────────┐ │ Accounts │ │ Categories │ │ Transactions │ │ Savings Goals │ │ Investments │ │
Bills │ └──────────────────────────────────────────┘ ↓ Daily transactions are recorded ↓ SQL Server stores the data ↓ ADO.NET
reads/writes data ↓ Web API exposes business operations ↓ MVC displays the result ↓ Dashboard calculates: Income Expenses
Savings Investments Remaining Plan vs Actual Goal Progress Upcoming Bills ↓ Monthly Review ↓ Create next month's plan
```

This is the complete learning path: first understand the database, then make SQL work, then connect it with ADO.NET, then expose it through Web API, then build the MVC interface, and finally connect everything into one working personal finance system.

19. Final Project Checklist

- ☐ Requirement finalized
- ☐ Database created
- ☐ Tables and relationships created
- ☐ Data dictionary documented
- ☐ ER diagram understood
- ☐ SQL CRUD practiced
- ☐ JOIN/report queries practiced
- ☐ Stored procedures created
- ☐ C# models created
- ☐ ADO.NET repository created
- ☐ Web API endpoints created
- ☐ MVC controllers and views created
- ☐ Monthly plan implemented
- ☐ Plan vs actual implemented
- ☐ Savings goals implemented
- ☐ Investments implemented
- ☐ Bills implemented
- ☐ Dashboard implemented
- ☐ Authentication implemented
- ☐ Validation and exception handling implemented
- ☐ Testing completed

The supplied reference document uses a structured Phase-2 approach—database objectives, core rules, data dictionary, relationship map, SQL strategy, delete strategy and implementation plan. This project-flow document carries that same planning discipline into the complete personal-finance development lifecycle. filecite turn0file0 L11-L29